

OSPF configuration and path selection

Follow-along: build a five-router network from nothing

Overview

Definition: Link-state routing and OSPF

In a *link-state* routing protocol, each router describes only what it is directly attached to (its own links, and which routers are on the far end of them) and floods that description to every other router. Each router therefore ends up holding an identical description of the whole network, and computes its own routes from it by running a shortest-path calculation over that description. *OSPF*, Open Shortest Path First, is the link-state protocol used inside a single organisation's network. This lab uses OSPF version 2, which carries IPv4 routes.

Five routers are wired together and none of them knows anything. No interface has an address, no router runs a routing protocol, and no host can reach any host but its own neighbours. By the end of Part 2 every one of the twenty ordered host pairs will be able to reach each other, and you will have typed every address and every line of configuration that makes it so. The routers run FRRouting, whose command line works the way a Cisco console does, so what you type here is close to what you would type on a commercial router.

The second half of the lab is about measuring the network you just built rather than assuming it is good. **OSPF** decides paths by adding up a cost per link. Every link in this lab comes up at the same cost, so here OSPF picks the route with the fewest routers on it rather than the fastest one. One link in this topology takes 100 ms to cross and it sits on the two-hop path, so the route OSPF picks between two of the hosts is seven times slower than the three-hop route around the other side. You find that by measuring, then move the traffic by changing one number at each end of one link. Part 4 closes a loose end in the configuration: the routers are sending OSPF onto the five links where the only other device is a host, and you stop that without losing the routes to those hosts.

This lab covers OSPF version 2 for IPv4 in a single area. It does not cover OSPFv3, dividing a network into several areas, or any protocol that carries routes between organisations, which is what the BGP labs are for.

Running this lab in the TUI

You drive the lab from the MiniLabs launcher. Clone it if the machine does not have it, and pull if it does, because handouts and lab scripts change together:

```
git clone https://github.com/nzRichie/MiniLab.git ~/MiniLab # first time
cd ~/MiniLab && git pull # already cloned
```

Start the launcher from that directory:

```
cd ~/MiniLab && ./minilabs
```

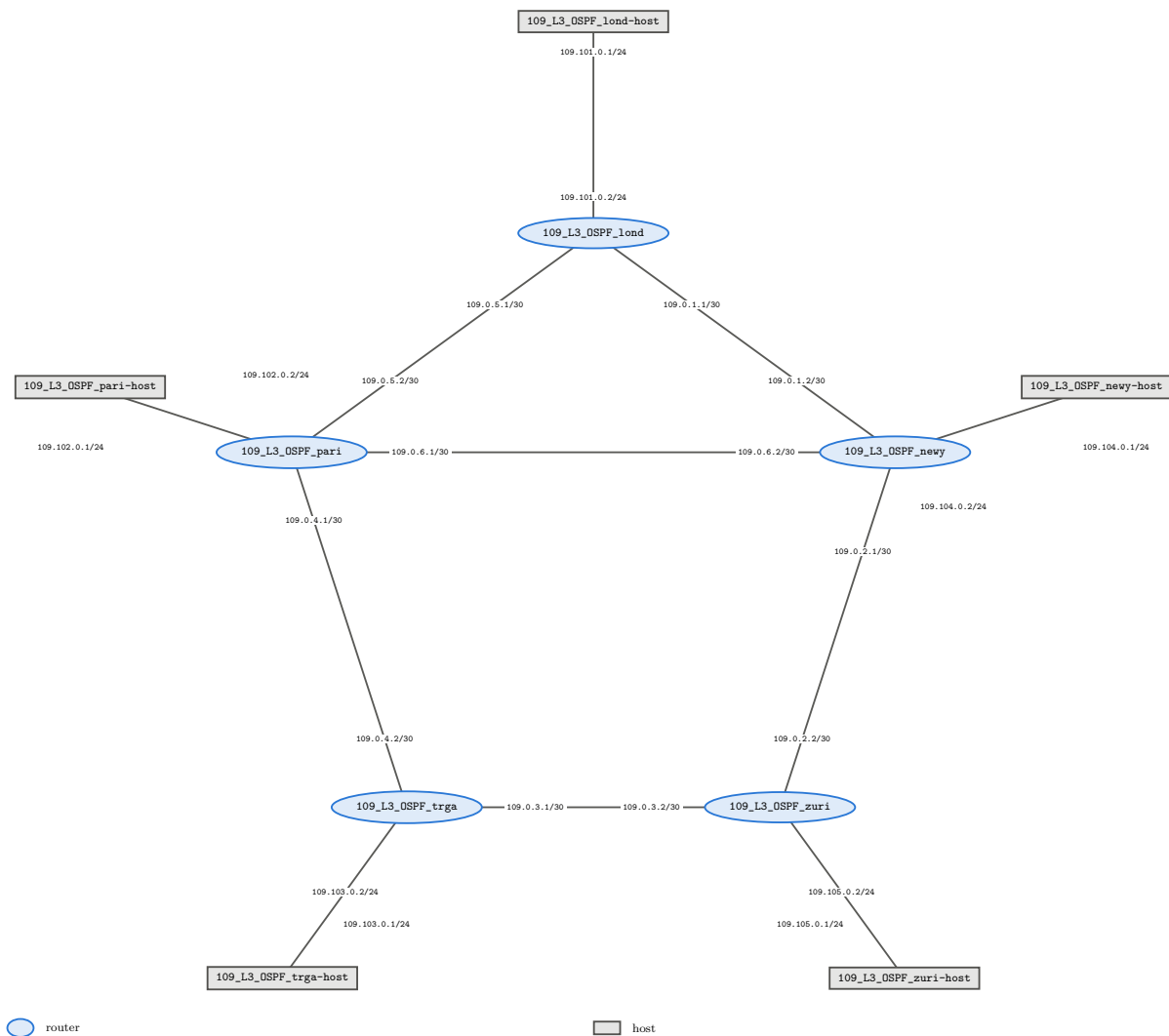
If it exits with a Docker error, follow the setup in the repository's `README.md` and run it again.

The root menu lists one category per row. Select **Network configuration**, then **OSPF configuration and path selection**, to reach this lab's actions:

- **Spawn** creates this lab's containers and wiring. Wait for the panel to turn green before you go on. The first spawn of a lab also builds its image, which takes a few minutes; later spawns take seconds. What it does *not* do is configure anything: every address in this lab is one you type.
- **Status** prints what each router currently has and what it can currently reach, so you read your own progress without shelling in. On a freshly spawned lab almost all of it is empty, which is correct. Status takes over the whole screen: the arrow keys scroll its output, and Enter on **Done** closes it.
- **Shell command for a node** lists every node (lond, pari, trga, newy, zuri, and lond-host, pari-host, trga-host, newy-host, zuri-host) and the `docker exec -it ...` line that opens a shell on it. Copy the line for the node a step below names into your own terminal. It holds the screen until Enter on **Done**, as Status does. A router line opens `vttysh`; a host line opens a plain shell.
- **Reset to baseline** puts every router and host back to the state they spawned in, without tearing the lab down. In this lab that means it removes everything you have configured, so use it when you want to start Part 1 again, not when you are stuck partway through Part 3.
- **Teardown** removes the lab.
- **Open handout** opens this document.

Spawn the lab now, then work through Part 1 in your own terminal. **Back** (or Esc) returns to the lab list.

The setup



Five routers named after cities, in a ring with one link across it. Each has one host behind it. PARI and NEWY have three router-to-router links each; LOND, TRGA and ZURI have two. Six links in all, so twelve interfaces face another router.

Every address below follows one scheme, so you can work any of them out rather than looking them up. Each router has a **location number**: LOND is 1, PARI is 2, TRGA is 3, NEWY is 4, ZURI is 5. From that number, with *Y* standing for it:

- the host subnet is 109.[100+Y].0.0/24, the host takes .1 and the router's own leg on that subnet takes .2;
- the loopback address is 109.[150+Y].0.1/32;
- each router-to-router link is a /30 out of 109.0.N.0/30, and on each one the router with the *lower* location number takes .1.

So NEWY, being location 4, has host subnet 109.104.0.0/24, its own leg on it at 109.104.0.2, and loopback 109.154.0.1/32.

Router	Loopback	Host subnet	Router-to-router interfaces
LOND (1)	109.151.0.1/32	109.101.0.0/24	port_NEWY 109.0.1.1, port_PARI 109.0.5.1
PARI (2)	109.152.0.1/32	109.102.0.0/24	port_TRGA 109.0.4.1, port_LOND 109.0.5.2, port_NEWY 109.0.6.1
TRGA (3)	109.153.0.1/32	109.103.0.0/24	port_ZURI 109.0.3.1, port_PARI 109.0.4.2
NEWY (4)	109.154.0.1/32	109.104.0.0/24	port_LOND 109.0.1.2, port_ZURI 109.0.2.1, port_PARI 109.0.6.2
ZURI (5)	109.155.0.1/32	109.105.0.0/24	port_NEWY 109.0.2.2, port_TRGA 109.0.3.2

A router's interface toward another router is named `port_` followed by that router's name, its interface toward its own host is `host`, and the host's interface toward the router is `router`. The interfaces already exist and are already up. What none of them has is an address.

Each host runs `ping`, `traceroute`, `tcpdump` and `curl`, and serves a line of text on port 80 of its own address that names which host it is. A `curl` that crosses the network therefore tells you which host received the request, which is a stronger check than `ping`: `ping` tells you something replied, and the line of text tells you the packet reached the host the addressing plan says owns that address.

Assessment

This lab carries fourteen graded questions, placed at the step that produces the observation each one asks about. Answer all fourteen in a separate document, numbered to match. Several ask for a number you measured; give the number, not a description of it.

Part 1: give every router an address

Nothing in this network is configured, so start by getting a shell on LOND. A router is configured through `vttysh`, FRRouting's command line, rather than through the shell:

```
# from your own terminal
docker exec -it 109_L3_OSPF_lond vtysh
```

Definition: Configuration mode

FRRouting's command line opens in *enable mode*, which has read-write access to the router and is where you run the commands that inspect it. `configure terminal` moves you into *configuration mode*, where the commands available are the ones that change the running configuration. Inside that, `interface port_NEWY` moves you again, into the context of one interface, so the commands you type next apply to that interface alone. `end` returns to enable mode from anywhere, and putting `no` in front of a command you have already given removes it. `show running-config` prints everything currently configured.

Enter **configuration mode** and give LOND its two router-to-router addresses:

```
# 109_L3_OSPF_lond: in the vtysh session you just opened
configure terminal
interface port_NEWY
  ip address 109.0.1.1/30
exit
interface port_PARI
  ip address 109.0.5.1/30
exit
```

Definition: Loopback address

A *loopback* is a virtual interface on a router that is not attached to any cable. It is up whenever the router is up, so it does not go down when a link does, and that is why it is used as a router's permanent identity: an address that identifies the router itself rather than one of its attachments. It is configured with a /32 prefix, because it names exactly one address and no subnet.

Now give LOND its leg on its own host subnet, and its **loopback address**. Still in configuration mode:

```
# 109_L3_OSPF_lond: same vtysh session
interface host
  ip address 109.101.0.2/24
exit
interface lo
  ip address 109.151.0.1/32
exit
end
show running-config
```

Do the same for PARI, TRGA, NEWY and ZURI, reading each address off the table above. That is twenty-two addresses in total across the five routers. Check your work on each router with **show interface brief** before you move on; a transposed digit here is the single most common reason the rest of the lab does not work, and it is far cheaper to find now.

Question 1. Once every router is addressed, get a shell on PARI and ping 109.0.4.2, 109.0.5.1 and 109.0.3.2 in turn. Give the result of each, then explain why the third behaves differently from the other two, naming what PARI holds that decides it.

Question 2. Each router-to-router link is a /30. State how many addresses that block contains, how many of them can be given to an interface, and what the other ones are reserved for. Then say why a /30 rather than a /24 is the sensible choice for a link with exactly two routers on it.

Now give each host its default route. A host is an ordinary Linux machine, so this is the **ip** command rather than **vttysh**:

```
# 109_L3_OSPF_lond-host: from your own terminal
docker exec -it 109_L3_OSPF_lond-host bash
ip route add default via 109.101.0.2
ip route show
```

Repeat on the other four hosts, each pointing at its own router's .2 address.

Question 3. From LOND's host, ping 109.101.0.2 and then 109.102.0.2. One succeeds and one does not. Give both results and explain what the default route did and did not achieve, given that every router interface in the network now has an address.

Part 2: turn on OSPF

Every router can now reach the routers it is directly cabled to, and nothing further. Each one holds routes only for the subnets it is itself attached to, and it has no way to learn about the rest. OSPF is what fills that gap, and the rest of this part is the configuration that turns it on.

Definition: Area, and the backbone area 0.0.0.0

OSPF divides a network into *areas*, so that a router in one area holds the detailed description of its own area and only a summary of the others. Every OSPF network has a backbone area numbered 0.0.0.0, and every other area must attach to it. A network small enough not to need the division puts everything in area 0.0.0.0, which is what this lab does: five routers is far below the size at which splitting into areas buys anything.

Definition: Router ID

A router's *router ID* is the 32-bit number that identifies it inside OSPF. It is written like an IPv4 address and it does not have to be one, but the usual practice is to set it to the router's loopback address, so that the identity in the database and the address you can reach the router at are the same number. If you do not set it, the router picks one from its own interfaces, and which one it picks depends on which addresses exist at the moment OSPF starts.

Turn OSPF on at LOND. Set its **router ID** from its loopback address, then name each subnet OSPF should run on:

```
# 109_L3_OSPF_lond: in vtysh
configure terminal
router ospf
  ospf router-id 109.151.0.1
  network 109.0.1.0/30 area 0.0.0.0
  network 109.0.5.0/30 area 0.0.0.0
  network 109.101.0.0/24 area 0.0.0.0
  network 109.151.0.1/32 area 0.0.0.0
exit
end
```

A **network** line does one thing: it selects the interfaces OSPF runs on, and it selects an interface whose own subnet sits inside the prefix on the line. Two things follow from that selection, and it is worth separating them. The router starts sending and listening for OSPF on the interface, and it advertises the subnet configured on that interface to the rest of the network. LOND's four lines therefore cover its two links, its host subnet, and its loopback. A subnet you leave out is one no other router will hear about.

Do the same on the other four routers, using each router's own loopback as its router ID, and one **network** line per subnet that router is attached to. PARI and NEWY have five lines each because they have three links; TRGA and ZURI have four, like LOND.

Definition: Neighbour and adjacency

Two OSPF routers on the same link find each other by sending *Hello* packets, and become *neighbours*. They then exchange their databases and reach the state **Full**, at which point they are *adjacent* and each holds the other's description of the network. An adjacency that is stuck below **Full** is the usual sign of a mismatch between the two ends, such as different subnet masks or different area numbers.

Definition: Designated router

On a link that could hold many routers, OSPF elects a *designated router* and a backup to it, and every other router on the link becomes adjacent to those two rather than to each of its neighbours in turn. That is $2n - 3$ adjacencies for n routers instead of the $n(n - 1)/2$ a full mesh needs: five routers on one link form seven adjacencies rather than ten. FRRouting treats an Ethernet interface as such a link by default, even when only two routers are on it, so the routers in this lab hold an election on every link and you will see states written as **Full/DR** and **Full/Backup**. The election has a waiting period before it completes, which is why an adjacency here can take up to 40 seconds to reach **Full** rather than appearing at once.

Give it a minute, then look at what LOND has found:

```
# 109_L3_OSPF_lond: in vtysh
show ip ospf neighbor
show ip route ospf
```

Question 4. Run `show ip ospf neighbor` on PARI and on ZURI. Give the number of neighbours each one reports, then explain what about the topology makes the two numbers differ. Your answer should refer to the figure, not to the output.

Definition: Link-state database and LSA

Each router's description of what it is attached to is carried in a *link-state advertisement*, or LSA. The collection of every LSA a router has received is its *link-state database*, and inside one area every router's database holds the same LSAs. `show ip ospf database` lists them. A router's own LSA lists each of its links and each subnet attached to it, which is more entries than it has neighbours: a host subnet and a loopback are attached to the router but have no other router on them.

Question 5. On TRGA, run `show ip ospf database router 109.153.0.1` to see the **link-state advertisement** TRGA originated about itself. Count the entries it lists, compare that with the number of neighbours TRGA has, and account for the difference by naming each entry that is not a neighbour.

Question 6. ZURI's loopback, 109.155.0.1, is on an interface with no cable attached to it. Explain how LOND ends up with a route to it, naming what ZURI puts into the database and what LOND does with it.

Once every router has all of its adjacencies, check the data plane rather than the control plane:

```
# 109_L3_OSPF_lond-host: from your own terminal
ping -c 3 109.105.0.1
traceroute -n 109.105.0.1
```

Question 7. On LOND, run `show ip route 109.155.0.1`, which is ZURI's loopback. Give the metric OSPF has assigned to that route, the next-hop address, and the interface LOND would send such a packet out of. State which router that next-hop address belongs to.

One experiment before moving on, because a **network** statement is the command in this lab that is easiest to get subtly wrong. On TRGA, remove the one covering its host subnet, wait ten seconds, and look at what changed from a router that is not TRGA:

```
# 109_L3_OSPF_trga: in vtysh
configure terminal
router ospf
no network 109.103.0.0/24 area 0.0.0.0
exit
end
```

```
# 109_L3_OSPF_lond: in vtysh
show ip route 109.103.0.0/24
```

Question 8. State what LOND now reports for that route, and predict whether TRGA's host can still reach its own gateway, 109.103.0.2. Check your prediction from TRGA's host. Explain the two results together: say what TRGA stopped doing and what it did not stop doing.

Put it back before going on, and confirm the route returns on LOND:

```
# 109_L3_OSPF_trga: in vtysh
configure terminal
router ospf
network 109.103.0.0/24 area 0.0.0.0
exit
end
```

Part 3: cost, and the path OSPF chooses

The network works. Whether it works *well* is a separate question, and this part is about measuring that rather than assuming it.

Definition: Cost

Every OSPF interface has a *cost*, a number representing how expensive it is to send a packet out of that interface. OSPF's shortest path is the path whose costs add up to the smallest total, not the path with the fewest routers on it. Cost applies to traffic *leaving* an interface, so the cost of a link in one direction is set on the interface at that end. In this lab every interface starts at cost 10, which you can read with `show ip ospf interface`.

Because every interface starts at the same **cost**, the total cost of a path here is 10 times the number of links on it, so at this point the cheapest path and the path with the fewest routers are the same path. That is a property of this network as it currently stands, not a rule about OSPF.

Measure what the network actually does. From each host, time the round trip to each of the other four:

```
# 109_L3_OSPF_lond-host: from your own terminal
ping -c 10 109.102.0.1
ping -c 10 109.103.0.1
ping -c 10 109.104.0.1
ping -c 10 109.105.0.1
```

Question 9. Measure the round-trip time from LOND's host to each of the other four hosts, and record the average of ten pings in each case. Give the four numbers, and name the pair that is worst.

Question 10. Run `traceroute -n` between the worst pair and give the address printed on each line of the output, naming the router or host each address belongs to. Then, using cost 10 on every interface, calculate the total cost LOND sees for each of the two routes to ZURI's host subnet 109.105.0.0/24, and say which one those totals make OSPF pick. Finally calculate the smallest whole-number cost that would have to be put on the NEWY-to-ZURI link for OSPF to prefer the other route, and show the comparison that gives you that number.

Now change it. The cost goes on the interface, not inside the `router ospf` section:

```
# 109_L3_OSPF_newy: in vtysh
configure terminal
interface port_ZURI
 ip ospf cost 100
exit
end
```

Wait ten seconds, then measure the round trip from LOND's host to ZURI's host again, and run `traceroute -n` from each of the two hosts toward the other.

One thing to know before reading that output. When a router sends back the reply `traceroute` uses to name a hop, it puts on that reply the address of the interface it would use to reach *you*, which is not necessarily the interface your packet arrived on. So on a path that is not symmetric, the addresses `traceroute` prints can name interfaces on the return path. The hop where the elapsed time jumps is the reliable signal, because that is where the delay was actually added.

Question 11. After setting the cost at the NEWY end only, give the new round-trip time from LOND's host to ZURI's host, and run `traceroute -n` from each of the two hosts toward the other. State the hop number at which the elapsed time jumps by about 100 ms in each of the two outputs. The two numbers differ, and the round-trip time is neither the one you recorded in Question 9 nor the value the other route alone would give. Explain both facts, using the rule that cost applies to traffic leaving an interface.

Now set the same cost at the other end:

```
# 109_L3_OSPF_zuri: in vtysh
configure terminal
interface port_NEWY
 ip ospf cost 100
exit
end
```

Definition: Equal-cost multipath

When two paths to the same destination have the same total cost, OSPF does not pick one: it installs both, and the router divides traffic between them. `show ip route` shows such a destination with more than one next hop. It is useful when you want the capacity of both paths, and awkward when you are trying to work out which way a particular packet went.

Before the cost change, PARI had two ways to reach ZURI's host subnet that cost the same: PARI to TRGA to ZURI, and PARI to NEWY to ZURI. Each crosses three interfaces on its way out (PARI's, then the middle router's, then ZURI's own `host` interface), so at cost 10 apiece each route totalled 30. Cost 100 now sits on NEWY's `port_ZURI`, which the route through NEWY crosses and the route through TRGA does not. Read what PARI installs for that subnet now:

```
# 109_L3_OSPF_pari: in vtysh
show ip route 109.105.0.0/24
```

Question 12. Run that command on PARI now, and state how many next hops it lists. Then explain what would have made it list two. One other tie like it existed before the cost change, between a different two routers: working from the topology figure with every interface at cost 10, name the router that had two equal-cost routes, name the host subnet it had them to, and give the two paths.

Part 4: keep OSPF off the host links

The network is finished and it is doing one thing it should not. Look at what has been arriving on a host all this time. On TRGA's host:

```
# 109_L3_OSPF_trga-host: from your own terminal
timeout 30 tcpdump -n -i router proto ospf
```

Question 13. Give the number of packets captured in those 30 seconds, the address they are sent to, and the interval between them. State what that destination address is for. Then, given that a host runs no routing protocol, say what the host does with each of these packets.

Those packets arrive because of a `network` statement you wrote in Part 2. Each router's host subnet needed one, or no other router would have learned a route to it. But a `network` statement selected that host-facing interface, and two things followed from the selection where only one was wanted: the subnet was advertised, which was the point, and the router also started speaking OSPF on the interface, which put its Hello packets onto a link whose only other device is a host.

Deleting the `network` statement is not the fix, because that would stop the subnet being advertised and every other host would lose its route to that one. The fix keeps the advertisement and stops the Hellos.

Definition: Passive interface

An interface marked *passive* is still advertised by OSPF as a subnet the router is attached to, but the router sends no Hello packets on it and forms no adjacency over it. It is the standard treatment for any interface that faces hosts rather than routers: the subnet stays in everyone's routing table, and the router stops speaking a routing protocol to machines that are not running one.

Mark the host link **passive** on all five routers. On LOND:

```
# 109_L3_OSPF_lond: in vtysh
configure terminal
interface host
  ip ospf passive
exit
end
```

Older documentation writes this as **passive-interface host** inside the **router ospf** section. FRRouting 9.1 still accepts that form, prints a deprecation notice, and stores it as **ip ospf passive** on the interface regardless, so the two end up as the same configuration.

Do the same on PARI, TRGA, NEWY and ZURI. Then repeat the capture on TRGA's host, and check that nothing else moved: the host pairs should all still reach each other, and every router should still hold all of its adjacencies.

```
# 109_L3_OSPF_trga-host: from your own terminal
timeout 30 tcpdump -n -i router proto ospf
```

```
# 109_L3_OSPF_lond: in vtysh
show ip ospf neighbor
show ip route 109.105.0.0/24
```

Question 14. Give the packet count from the second capture. Then state what LOND's two commands report for the number of neighbours and the route to ZURI's host subnet, and explain why marking an interface passive changed the first number and neither of the other two. Your answer should name the two things that follow from a **network** statement selecting an interface.

Checks: what “done” means

Run **Status** from the TUI. On a finished lab it reports:

- all five routers addressed, each with its loopback;
- 12 of 12 router-to-router adjacencies Full;
- cost 100 at both ends of the NEWY-to-ZURI link;
- LOND's host to ZURI's host at roughly 30 ms, through PARI and TRGA;
- 20 of 20 ordered host pairs reaching each other, each replying with its own name;
- zero OSPF packets on all five host links.

If you want to start Part 1 again, **Reset to baseline** removes everything you have configured and returns the lab to the state it spawned in. It does not remove the per-link delays, which belong to the wiring rather than to your configuration.

Safety note

Every container in this lab is wired with point-to-point virtual Ethernet pairs that exist only inside the containers themselves. There is no bridge to the host's network, no route off this machine, and no address in this lab is reachable from anywhere else. The addresses used here sit in 109.0.0.0/8, which is allocated on the real internet; nothing in this lab can reach it, and nothing on it can reach this lab.

Teardown

When you are finished, remove the lab: pick **Teardown** from the menu, or run `scripts/teardown.sh` directly. It deletes this lab's containers and nothing else.